COESIONE ITALIA 21-27 PIEMONTE

Cofinanziato dall'Unione europea

REGIONE PIEMONTE

DOCENTE Seyedhossein Javadizavieh

UNITA' FORMATIVA Amazon Web Services - Introduzione e utilizzo

ARGOMENTO Lezione 07 - Dockerfile, networking e Kubernetes

intervento realizzato da

ITS ACADEMY ICT·PIEMONTE

Agenda

1. Gestione con Dockerfile
2. Networking nel container
3. Kubernetes
4. Configurazione, persistenza e monitoring
5. Mini-exercises
6. Lab + cleanup
7. Wrap-up + suggested links

Obiettivi

- Descrivere la funzione di un Dockerfile.
- Riconoscere i principi del networking nel container.
- Collegare Kubernetes a configurazione, persistenza e monitoring.

Dall'immagine ricevuta all'applicazione dichiarata

Nella lezione 6 abbiamo eseguito un'immagine esistente. Ora dobbiamo consegnare una pagina nostra, collegare container e conservare un dato oltre la loro sostituzione.

Tre artefatti distinti: Dockerfile per costruire, configurazione di esecuzione per collegare, volume per conservare. Kubernetes aggiunge un controllo continuo sullo stato desiderato.

Il progetto minimo, prima della build

Creare una cartella dedicata `lab07` con questi tre file:

```
lab07/
  Dockerfile
  index.html
  .dockerignore
```

Contenuto di `index.html`: `<h1>LAB07-OK</h1>`. In `.dockerignore` inserire, su righe separate, `.git`, `.env`, `.aws` e `*.log`.

Il contesto è l'insieme dei file disponibili alla build: non usare la cartella personale come contesto.

Dockerfile completo: una pagina servita da Nginx

File `lab07/Dockerfile`:

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

`COPY` inserisce la pagina nell'immagine. `CMD` mantiene Nginx in primo piano: se il processo principale termina, il container si ferma. `EXPOSE` documenta una porta, non la pubblica.

Base mobile per la dimostrazione: registrare il digest. Questo esempio essenziale non è una configurazione di hardening per produzione.

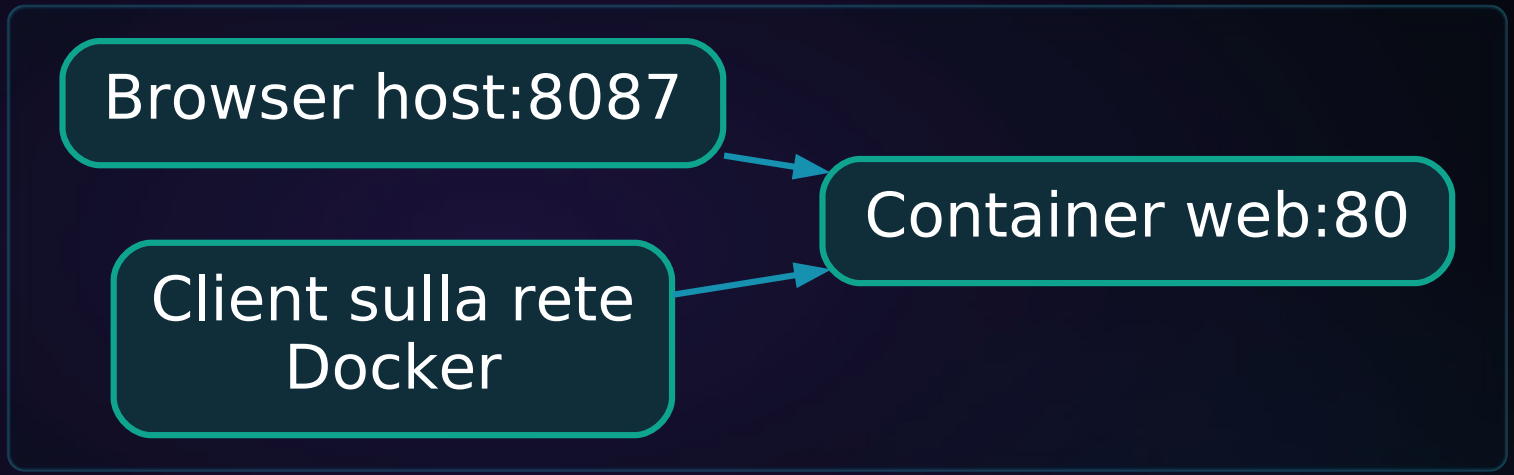
Costruire e verificare dal computer

Dal terminale aperto nella cartella `lab07`:

```
docker build -t introaws07-web:1.0 .
docker network create introaws07-net
docker run -d --name introaws07-web --network introaws07-net -p 127.0.0.1:8087:80
introaws07-web:1.0
```

Aprire `http://127.0.0.1:8087`: deve comparire **LAB07-OK**. La stessa sintassi Docker funziona in PowerShell, Bash e zsh; non servono continuazioni di riga specifiche della shell.

Due percorsi verso la stessa applicazione




```
docker run --rm --network introaws07-net nginx:alpine wget -qO- http://introaws07-web:80
```

Il client usa il nome del container e la porta **80**, non la porta host 8087.
Dentro il client, `127.0.0.1` indica il client stesso, non il server web.

Verificare la persistenza con una sostituzione

```
docker volume create introaws07-data
docker run --rm --mount source=introaws07-data,target=/data nginx:alpine sh -c 'echo
LAB07-PERSISTE > /data/prova.txt'
docker run --rm --mount source=introaws07-data,target=/data nginx:alpine cat
/data/prova.txt
```

Il primo container è già stato rimosso quando parte il secondo. La lettura di **LAB07-PERSISTE** dimostra che il dato appartiene al volume, non al layer scrivibile del primo container.

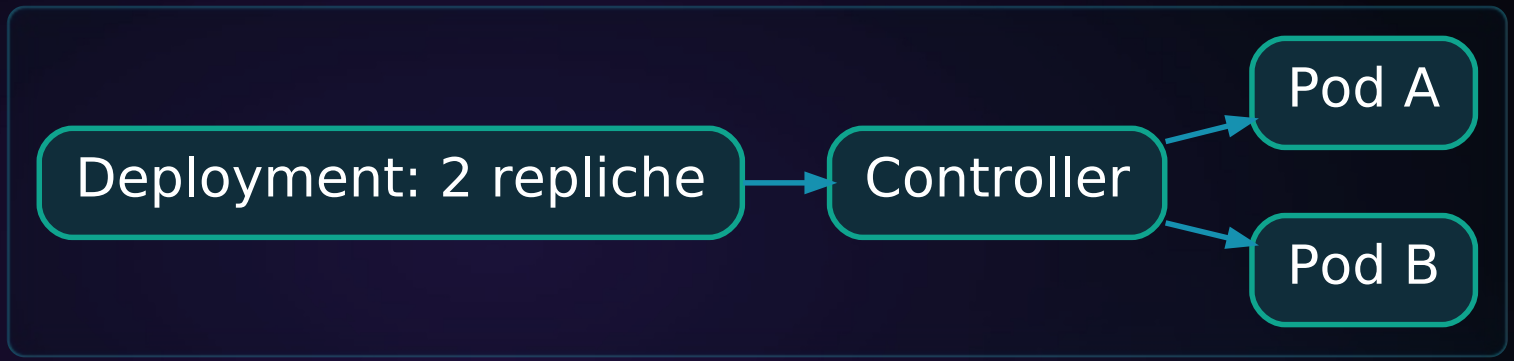
Un parametro serve solo se qualcuno lo legge

```
docker run --rm -e APP_ENV=didattica nginx:alpine printenv APP_ENV
```

Output: `didattica`. Questo dimostra l'iniezione della variabile, **non** una modifica automatica alla pagina Nginx: l'applicazione deve usare esplicitamente il parametro.

In Kubernetes ConfigMap conserva configurazione non sensibile. Un Secret non diventa sicuro per il solo fatto di usare base64: servono controllo degli accessi e protezione adeguata.

Kubernetes: dichiarare, osservare, riconciliare



Il controller confronta stato desiderato e osservato. Se un Pod viene perso, cerca di ripristinare le repliche. Non ripara automaticamente un'immagine errata, un database corrotto o un volume non disponibile.

Service: stabilità del punto di accesso



Il Service seleziona i Pod tramite etichette e offre un endpoint stabile. `port` è la porta del Service; `targetPort` quella dell'applicazione. Un ClusterIP non pubblica da solo il servizio su Internet.

Un frammento YAML da leggere prima di applicare

Parte del template di un Deployment; il manifesto completo è nel lab:

```yaml
containers:
  - name: web
    image: nginx:alpine
    ports:
      - containerPort: 80
    readinessProbe:
      httpGet:
        path: /
        port: 80
      initialDelaySeconds: 2
      periodSeconds: 5
```

Readiness negativa esclude il Pod dagli endpoint pronti; non equivale alla liveness e non ordina da sola un riavvio.

Persistenza Kubernetes: richiesta e disponibilità

Una **PVC** richiede storage. Una StorageClass e un provisioner possono soddisfarla; non è garantito che ogni cluster disponga di questi componenti.

Il Pod monta il volume associato alla claim. La sostituzione del Pod non richiede la cancellazione della PVC.

Eliminare la claim può invece provocare la cancellazione dello storage, secondo la reclaim policy. Persistenza non significa backup: il recupero va progettato e provato separatamente.

Osservabilità: partire dalla domanda

Domanda	Evidenza	Interpretazione
Il processo è attivo?	Stato container/Pod	Non prova una risposta corretta
La richiesta arriva?	Access log	Collegare URL, stato e orario
Può ricevere traffico?	Readiness / endpoint	Separare avvio e disponibilità
Mancano risorse?	Metriche CPU/memoria	Servono strumenti di raccolta

`kubectl top` richiede un servizio metriche disponibile. Log e stato non diventano metriche di utilizzo solo perché compaiono nello stesso terminale.

Leggere i log dopo una richiesta

```
curl.exe --noproxy "*" -i http://127.0.0.1:8087/
docker logs --tail 20 introaws07-web
docker inspect --format '{{.State.Status}}' introaws07-web
```

Windows PowerShell: `curl.exe`; Linux/macOS: `curl`. Cercare nel log la GET e il codice 200. Non cercare `.State.Health.Status`: il Dockerfile di questo caso non definisce un HEALTHCHECK.

Confine del laboratorio: locale, non EKS

La parte Docker è locale. La parte Kubernetes richiede un cluster **locale già predisposto**, con contesto verificato; in sua assenza si analizzano manifest e traccia sintetica.

Non creiamo EKS, nodi EC2, ruoli IAM o load balancer AWS. `kubectl --dry-run=client` non è una garanzia di lavoro offline: può richiedere discovery e validazione del server.

Il cleanup elimina soltanto container, rete, volume e namespace dedicati a questo esercizio.

Mini-esercizio Sessione 07 - Networking e persistenza

Un’app deve raggiungere un servizio interno e conservare dati. Che cosa va configurato?

- A) Soltanto il nome dell’immagine
- B) Soltanto il colore dei log
- C) Networking e persistenza



Mini-esercizio Sessione 07 - Networking e persistenza (risposte)

Un’app deve raggiungere un servizio interno e conservare dati. Che cosa va configurato?

- C) Networking e persistenza

Risposte attese:

- **Corretta: C** - Connettività e storage risolvono requisiti distinti.
- A è errata: Il nome dell’immagine non crea rete o storage.
- B è errata: Il colore dei log non conserva dati.

Mini-esercizio Sessione 07 - Contesto di build

Come si limita un contesto Docker troppo ampio?

- A) Copiando sempre l’intero filesystem
- B) Inserendo le credenziali nell’immagine
- C) Con `.dockerignore` e copie mirate



Mini-esercizio Sessione 07 - Contesto di build (risposte)

Come si limita un contesto Docker troppo ampio?

- C) Con `.dockerignore` e copie mirate

Risposte attese:

- **Corretta: C** - Esclusioni e COPY mirati riducono dimensione e rischio.
- A è errata: Copiare tutto amplia il contesto.
- B è errata: Le credenziali non devono essere incorporate.

Mini-esercizio Sessione 07 - Rete interna dei container

Due container devono comunicare senza esporre il servizio all’esterno. Quale soluzione è adatta?

- A) Pubblicare obbligatoriamente ogni porta sull’host
- B) Usare file locali senza rete
- C) Una rete condivisa con risoluzione per nome



Mini-esercizio Sessione 07 - Rete interna dei container (risposte)

Due container devono comunicare senza esporre il servizio all’esterno. Quale soluzione è adatta?

- C) Una rete condivisa con risoluzione per nome

Risposte attese:

- **Corretta: C** - La rete condivisa abilita la comunicazione interna tra servizi.
- A è errata: La pubblicazione sull’host non è necessaria per il solo traffico interno.
- B è errata: File locali non sostituiscono la comunicazione di rete.

Mini-esercizio Sessione 07 - Storage persistente

Un Pod viene sostituito e i dati locali scompaiono. Che cosa serve?

- A) Più log nel container
- B) Un nome immagine diverso
- C) Storage persistente dichiarato separatamente



Mini-esercizio Sessione 07 - Storage persistente (risposte)

Un Pod viene sostituito e i dati locali scompaiono. Che cosa serve?

- C) Storage persistente dichiarato separatamente

Risposte attese:

- **Corretta: C** - Lo storage persistente ha un ciclo di vita distinto dal container.
- A è errata: I log non preservano i dati applicativi.
- B è errata: Cambiare immagine non garantisce persistenza.

Lab/Exercises

- Costruire un'immagine descritta da Dockerfile.
- Configurare networking tra container.
- Rappresentare configurazione, persistenza e monitoring in Kubernetes.

Cleanup: rimuovere container, immagini, reti e risorse Kubernetes create per l'esercitazione.

Link: Dockerfile, networking e Kubernetes

Wrap-up Lesson 07

- Il Dockerfile rende ripetibile la costruzione di un'immagine.
- Il networking collega i container secondo regole esplicite.
- Kubernetes coordina configurazione, persistenza e monitoring del workload.

Lesson 07 - Suggested links

- https://docs.docker.com/reference/dockerfile/
- https://kubernetes.io/docs/concepts/cluster-administration/networking/
- https://kubernetes.io/docs/concepts/configuration/
- https://kubernetes.io/docs/concepts/storage/
- https://kubernetes.io/docs/concepts/cluster-administration/monitoring/